

Smart Library Seat Management System

On-Demand Seating Allocation, Real-time Tracking, and Administrative Analytics Platform

Ayush Singh Sikarwar, CSED

Roll No: 1024030271

Satvik Ganda, CSED

Roll No: 1024030158

Aryan Bansal, CSED

Roll No: 1024030204

Rishit Parnami, CSED

Roll No: 1024030270

September 8, 2026

Project Proposal for UCS503P – Software Engineering Lab

Thapar Institute of Engineering and Technology

1 Higher-order Goal

Real-world impact

The Smart Library Seat Management System is envisioned to transform the historically chaotic and high-friction process of locating and securing physical study spaces within university and public libraries. Currently, students must physically survey multiple floors and wings to locate a vacant seat, often encountering seats that are pseudo-occupied by unattended belongings.

This project aims to introduce a data-driven, fair, and highly transparent digital layer over the physical library infrastructure. The engineering objective is to build a robust software ecosystem that bridges real-time physical occupancy with a digital twin interface. This involves coordinating large-scale concurrent user access, enforcing time-bound automated state transitions (such as temporary-away limits), and providing library administrators with actionable utilization analytics. The project directly addresses a universally recognized pain point in academic institutions, offering immense real-world utility while serving as a comprehensive vessel for demonstrating advanced Software Engineering principles.

2 Problem Statement

In peak academic periods—such as mid-semester evaluations and end-of-term examinations—library seating becomes an acute scarcity. The absence of a centralized digital management system leads to several compounding operational failures:

- **High Search Friction:** Students waste significant cognitive bandwidth and time (often 10–20 minutes) navigating vast library floors simply to assess availability.
- **Unfair Resource Blocking (Ghosting):** A pervasive culture of 'bag-blocking', where students leave their belongings to reserve a seat while they attend classes, go for lunch, or take prolonged breaks. This artificially restricts the library's effective capacity.

- **Lack of Real-time Visibility:** Library staff have no empirical mechanism to distinguish between a genuinely occupied seat and one that has been abandoned, making policy enforcement virtually impossible.
- **Suboptimal Infrastructure Planning:** Without historical analytics regarding peak hours, zone preferences (e.g., silent zones vs. collaborative pods), and seating turnover rates, administrators cannot optimize the physical layout or allocate resources effectively.

3 Proposed Solution

Software Proposition

The proposed system is a comprehensive, responsive web-based platform that acts as an authoritative ledger for library seating. It connects end-users (students/patrons) with administrative oversight through a unified, state-driven workflow.

The core modules of the solution include:

- **Interactive Digital Floor Map:** A dynamic SVG or Canvas-based rendering of the library layout, providing color-coded, real-time statuses for every individual seat across all floors and zones.
- **Smart Seat Finder & Recommendation Engine:** An algorithmic matching system that allows users to filter seats based on explicit parameters (e.g., proximity to charging ports, proximity to natural light, quiet vs. discussion zones, wheelchair accessibility).
- **The 'Temporary Away' Subsystem:** A highly regulated mechanism allowing students to momentarily lock their seat (e.g., for a 30-minute lunch break). This initiates a server-side countdown timer.
- **Automated Fair-Use Enforcement:** If the 'Temporary Away' timer expires without the student checking back in, the system automatically revokes the session and broadcasts the seat's availability to all connected clients.
- **Administrative Analytics Dashboard:** A secure portal for library staff providing live heatmaps of occupancy, daily peak utilization graphs, and the ability to manually override seat statuses (e.g., marking a seat for maintenance).

4 Core Workflow and State Model

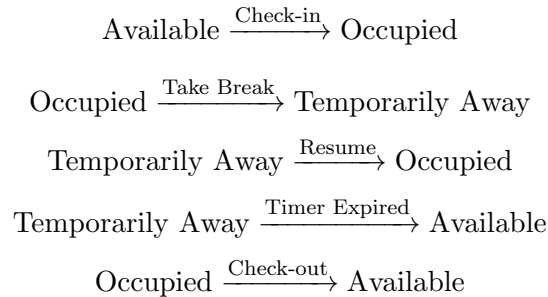
The lifecycle of a library seat and a user's session is governed by a strict finite state machine.

4.1 User Journey

1. **Authentication:** The user logs into the platform using Single Sign-On (SSO) or institutional credentials.
2. **Discovery:** The user views the live map or queries the Smart Finder.
3. **Check-in (Occupancy):** The user selects an 'Available' seat and confirms occupancy. A session token is generated.
4. **Pause (Temporary Away):** If a break is required, the user triggers the 'Away' state. A strict countdown (e.g., 30 mins) begins on the backend.
5. **Resume / Expiration:** The user returns and resumes the session. Alternatively, the timer hits zero, triggering an automated release protocol.
6. **Check-out:** The user intentionally ends their session upon leaving the library.

4.2 Mathematical State Transitions

The system enforces the following state transitions for any given seat entity S :



Exceptional states such as *Maintenance* or *Reserved (for events)* will bypass standard user flows.

5 Why This is Suitable for UCS503

This project is intentionally designed to exceed the complexity of a standard CRUD (Create, Read, Update, Delete) application. It introduces challenges typical of modern distributed systems:

- **Concurrency and Race Conditions:** Handling scenarios where two students attempt to book the last available seat simultaneously requires robust database locking mechanisms and transactional integrity.
- **Real-Time State Synchronization:** Pushing instantaneous state changes to hundreds of connected mobile clients necessitates efficient use of WebSockets or Server-Sent Events (SSE).
- **Background Job Processing:** Managing thousands of asynchronous countdown timers reliably requires decoupling timer logic from the main request-response cycle.
- **Architectural Scalability:** The architecture must survive traffic spikes common during university exam weeks.

6 Mapping to Course Learning Outcomes (CLOs)

CLO	How the Project Demonstrates the CLO
CLO 1: Process Models	An Agile/Scrum software development lifecycle will be adopted. The project is inherently modular (Auth, Layout Engine, State Engine, Analytics), making it ideal for iterative two-week sprints. The iterative process allows for early validation of the seat-booking mechanism before investing in complex analytics.
CLO 2: Requirements Engineering	Requirements will be systematically elicited from multiple stakeholders (students, librarians, IT staff). These will be formalized in an SRS document, translated into detailed use-case, activity, sequence, and state-machine diagrams to unambiguously define the 'Temporary Away' edge cases.

CLO Software Design	3:	The system will rigidly adhere to the MVC architecture. Principles of low coupling and high cohesion will be demonstrated by separating the Web-Socket broadcasting service from the core database repositories. ER diagrams will model complex relationships between Zones, Seats, Users, and Historical Sessions.
CLO Testing Strategies	4:	The automated timer and state transitions provide a highly testable business logic layer. Test-Driven Development (TDD) will be applied to the concurrency locks. The project will document Unit Testing (for pricing/timers), Integration Testing (for API endpoints), and Load Testing (to simulate exam-week traffic).
CLO Project Management	5:	The team will utilize Agile boards (Jira/GitHub Projects), story pointing, and velocity tracking. CI/CD pipelines will be established for automated linting, testing, and deployment to staging environments, demonstrating modern DevOps practices.

7 Requirements Engineering

7.1 Principal Stakeholders

- **Students/Patrons:** Primary end-users requiring fast, mobile-friendly access to seat availability.
- **Library Administrators:** Operational staff requiring oversight, policy control, and reporting.
- **Maintenance Staff:** Personnel who resolve hardware issues (e.g., broken chairs, faulty sockets) reported through the system.

7.2 Functional Requirements (FR)

1. The system must provide a visual representation of all library floors and seating zones.
2. The system must restrict a single authenticated user to one active seat session at a time.
3. The system must execute a server-side timer (configurable by admins, default 30 mins) when a user enters the 'Away' state.
4. The system must automatically release a seat and notify the user if their 'Away' timer expires.
5. The system must allow users to report maintenance issues associated with specific seats.
6. The system must generate aggregated daily, weekly, and monthly occupancy reports.

7.3 Non-Functional Requirements (NFR)

- **Latency:** Real-time state changes (e.g., a seat becoming available) must be broadcast to all active clients within 500 milliseconds.
- **Availability:** The system must guarantee 99.9% uptime during library operational hours.
- **Responsive Design:** The user interface must be fully functional and visually coherent on mobile devices, as over 80% of student interactions will occur via smartphones.
- **Security:** Passwords (if not using SSO) must be salted and hashed (e.g., bcrypt). API endpoints must be protected against CSRF and SQL Injection.

8 UML and System Modelling

To ensure architectural clarity before implementation, the following models will be generated:

- **Use-Case Diagram:** Delineating the interaction boundaries between Students, the System Timer, and Library Admins.
- **Activity Diagram:** Mapping the intricate logic flow of the 'Temporary Away' process, including grace periods and penalty logging.
- **Sequence Diagram:** Visualizing the chronological flow of messages during a concurrent booking conflict, demonstrating how the database lock resolves the race condition.
- **Entity-Relationship (ER) Diagram:** Defining normalized schemas for 'Users', 'Library_Zones', 'Seats' (with foreign keys to Zones), 'Active_Sessions', and 'Session_History'.

9 Software Architecture and Design Principles

A robust three-tier Service-Oriented Architecture (SOA) is proposed:

- **Presentation Tier:** A Single Page Application (SPA) built using React/Vue, ensuring smooth client-side routing and minimal DOM repaints when seat statuses change.
- **Application Tier:** A RESTful API built with Node.js/Express or Python/FastAPI. A dedicated WebSocket server will handle bidirectional real-time communication.
- **Data Tier:** A relational database (e.g., PostgreSQL) will ensure ACID compliance, which is critical for handling concurrent seat reservations safely. Redis may be introduced for caching layout configurations and managing high-speed timer expirations.

Design Principles Applied:

- **Separation of Concerns:** The visual rendering engine for the SVG map will be entirely decoupled from the business logic handling session timers.
- **High Cohesion:** All logic pertaining to user penalties or historical logging will reside strictly within the Analytics/Reporting modules.

10 Green Software Engineering

In alignment with modern sustainability practices, the system will minimize computational and network waste:

- **Network Optimization:** Moving away from aggressive client-side HTTP polling (which drains mobile batteries and clogs networks) in favor of lightweight WebSocket event payloads (pushing only the ID of the changed seat, not the entire map).
- **Asset Caching:** The heavy SVG floor maps and static assets will be aggressively cached via service workers and edge CDNs.
- **Dark Mode Default:** The UI will feature a true-black dark mode to significantly reduce power consumption on student OLED smartphone screens.

11 Agile Development Plan

Development will follow a structured 6-sprint cycle:

Sprint	Deliverable & Focus
Sprint 1	<i>Foundation:</i> Requirements finalization, SRS drafting, UI wireframing, and Database schema provisioning.
Sprint 2	<i>Core API:</i> User authentication (JWT/SSO), basic REST endpoints for CRUD operations on Seats and Zones.
Sprint 3	<i>Interactive Frontend:</i> Integration of the SVG floor map. Students can click seats to view static metadata.
Sprint 4	<i>The State Engine:</i> Implementation of check-in, check-out, and the critical 'Temporary Away' timer logic. Integration of WebSockets for live UI updates.
Sprint 5	<i>Admin & Analytics:</i> Building the librarian dashboard, generating occupancy heatmaps, and implementing maintenance-reporting workflows.
Sprint 6	<i>Hardening:</i> Extensive Load testing, edge-case debugging, Green Software optimizations, and final deployment to production staging.

12 Evaluation Criteria & Key Performance Indicators

The success of the software will be evaluated quantitatively during a pilot phase:

- **Seat Search Latency:** Reduction in the average time a student takes to find a seat (Target: < 2 minutes).
- **Abandonment Recovery Rate:** The percentage of seats successfully identified as abandoned and automatically released back into the pool.
- **System Uptime & Latency:** API responses must remain under 200ms even under simulated peak loads of 1,000 concurrent users.
- **Defect Density:** Achieving a high pass rate (> 95%) on the automated test suite before the final release.

13 Future Scope and Extensions

While the Minimum Viable Product (MVP) relies on software-enforced timers and user honesty, the architecture is designed to accommodate future physical integrations:

- **IoT Sensor Integration:** Deploying inexpensive PIR (Passive Infrared) or pressure sensors on seats to automatically detect physical occupancy and reconcile it with the digital state.
- **RFID Integration:** Linking the system with library turnstiles to automatically check out students who leave the building without manually ending their session.
- **Predictive Analytics:** Using Machine Learning models on historical data to predict peak hours and suggest optimal arrival times to students.

14 Summary

The Smart Library Seat Management System is an ambitious, highly practical project that addresses a pervasive operational flaw in academic libraries. By combining real-time web technologies, strict state management, and data analytics, the project delivers immense utility. For the context of UCS503, it perfectly encapsulates the necessity of rigorous software engineering—requiring meticulous requirements analysis, scalable architectural design, agile project management, and comprehensive testing methodologies to successfully execute.